

Plano de Teste – API Restful-Booker

1. Objetivo

Definir a estratégia e os cenários de teste para a API *Restful-Booker*, com foco na validação dos fluxos de autenticação e gestão de reservas, garantindo conformidade com a documentação oficial e comportamento esperado.

2. Escopo

Este plano de testes contempla as seguintes funcionalidades:

- Autenticação de usuários
 - Gestão de reservas (CRUD)
 - Filtros e buscas
 - Tratamento de erros e validação de dados
-

3. Estratégia de Testes

Serão executados testes funcionais do tipo:

- Testes positivos (fluxos válidos)
 - Testes negativos (erros e validações)
 - Testes de API com base em requisições HTTP (GET, POST, PUT, PATCH, DELETE)
-

4. Ambiente de Teste

- API: Restful-Booker
 - Formato de dados: JSON
 - Ferramentas sugeridas: Postman / Insomnia / REST Client
-

5. Casos de Teste

5.1 Autenticação – Endpoint `/auth`

ID	Caso de Teste	Descrição	Entrada	Resultado Esperado
CT01	Gerar token com sucesso	Validar login com credenciais válidas	{ "username": "admin", "password": "password123" }	Status 200 e retorno de token
CT02	Login inválido	Validar comportamento com credenciais incorretas	Usuário/senha inválidos	Status 200 com mensagem "Bad credentials"

5.2 Gestão de Reservas – Endpoint `/booking`

ID	Caso de Teste	Descrição	Método	Resultado Esperado
CT03	Criar reserva	Criar uma reserva com payload válido	POST	Status 200 e retorno com <code>bookingid</code>
CT04	Consultar reserva	Consultar dados de uma reserva existente	GET <code>/booking/:id</code>	Status 200 com dados corretos
CT05	Atualizar reserva	Atualizar dados da reserva (total e parcial)	PUT / PATCH	Status 200 e dados atualizados
CT06	Excluir reserva	Remover reserva existente	DELETE	Status 201 conforme padrão da API

Observação: Requer autenticação via Cookie: `token={{token}}`.

5.3 Filtros e Buscas

ID	Caso de Teste	Descrição	Endpoint	Resultado Esperado
CT07	Filtrar por nome	Filtrar reservas por <code>firstname</code> e <code>lastname</code>	<code>/booking?firstname=Sally&lastname=Brown</code>	Status 200 e lista de IDs correspondentes
CT08	Filtrar por data	Filtrar reservas por <code>check-in</code>	<code>/booking?checkin=2023-01-01</code>	Status 200 com reservas válidas

5.4 Tratamento de Erros e Validações

ID	Caso de Teste	Descrição	Ação	Resultado Esperado
CT09	Recurso inexistente	Consultar ID inválido	GET	Status 404 Not Found
CT10	Tipo de dado inválido	Enviar <code>totalprice</code> como string	POST/PUT	Status 400 ou erro de tipo
CT11	Acesso sem token	Alterar/excluir sem autenticação	PUT/DELETE	Status 403 Forbidden
CT12	Campo obrigatório ausente	Criar reserva sem <code>lastname</code>	POST	Status 400 ou 500

6. Critérios de Aceitação

- Todos os endpoints respondem conforme esperado
- Nenhum erro crítico (5xx inesperado)
- Dados retornados corretamente estruturados
- Autenticação obrigatória respeitada nas operações protegidas

7. Riscos e Observações

- A API apresenta comportamentos não padronizados (ex: erro de login retorna status 200)
- Algumas validações podem depender da implementação atual (ex: status 400 vs 500)